

Coverity 2026.6

Multiplatform Deployment Guide

Architecture, sizing, installation, security, migration, and operations for Cloud Native Coverity across AWS, Azure, Google Cloud, Red Hat OpenShift, and customer-managed Kubernetes.

Release baseline: 2026.6.0 · Document revision: 29 September 2026

Status: engineering reference — site validation required before production use.

How to use this guide: select a deployment profile, prepare the shared prerequisites, follow one provider chapter, then complete installation and acceptance testing. Provider and feature values are layered deliberately; do not combine every example into one deployment.

Contents

1. [Scope, support, and release controls](#)
2. [Platform architecture](#)
3. [Sizing and capacity planning](#)
4. [Shared preparation and image supply](#)
5. [Database, secrets, and certificate trust](#)
6. [Shared Helm configuration](#)
7. [AWS EKS](#)
8. [Microsoft Azure AKS](#)
9. [Google Kubernetes Engine](#)
10. [Red Hat OpenShift](#)
11. [Customer-managed Kubernetes and dependencies](#)
12. [Routing, TLS, and network access](#)
13. [Analysis workers, caching, and toolkits](#)
14. [High availability and database read replicas](#)
15. [AI-Assisted Triage](#)
16. [Validation and installation](#)
17. [Acceptance testing and integrations](#)
18. [Migration, upgrades, and recovery](#)
19. [Operations and troubleshooting](#)
20. [Evaluation and disconnected environments](#)
21. [References and validation record](#)

1. Scope, support, and release controls

This guide covers the CNC Helm deployment model. It does not replace the classic Coverity installer guide or imply support for every Kubernetes distribution, managed database variant, or combination of optional features.

Deployment profiles

Profile	Components	Use
Connect-only	Connect, setup jobs, administrative tools, PostgreSQL.	Analysis remains on build agents or developer systems.
Connect + Scan Services	Connect, Scan Service, Storage Service, certificate/cleanup jobs, analysis workers, PostgreSQL, object storage.	Thin Client or full-client submission to in-cluster analysis.
Cached analysis	Previous profile plus Cache Service, Redis, and a separate cache bucket/container.	Reuse eligible unchanged analysis inputs and results.
Connect HA	Multiple Connect pods, optional dedicated commit servers, optionally Pgpool and database read replicas.	Availability and concurrent workload distribution.
AI-Assisted Triage	Suggestion API, workers, PostgreSQL, RabbitMQ, artifact storage, authentication/encryption keys, approved LLM.	Optional beta capability; separate security approval required.

1.1 Release pinning

- Pin the CNC chart, application images, analysis toolkits, third-party charts, controllers, and CRDs to reviewed versions.
- Do not use `latest`, `main`, `latest`, or a moving branch as a production release reference.
- Record archive checksums and deployed image digests. Preserve packaged chart dependencies.
- A future “latest release” edition requires a fresh support and upgrade review; changing one version variable is not sufficient.

1.2 Published support information

The supplied 2026.6 support table reports the following values. These are source-reported entries, not a independently tested compatibility certification for this guide.

Item	Source-reported value	Deployment interpretation
Kubernetes	1.34–1.36	Confirm release availability and the provider-supported combination.
Helm	4.0.0 or later	Use the release support requirement, not lower Helm versions in older examples.
OpenShift	4.19	Requires clarification against the Kubernetes range; do not combine unrelated table rows.
PostgreSQL	14.22–18.x	Confirm the exact major/minor and deployment model before provisioning or migrating.
Redis	6.0 or greater	Provider restrictions still apply; version alone does not establish compatibility.
AWS / GCP / Azure	1.34.9 / 1.35.5 / 1.34.8	Reported tested platform versions; verify actual provider offerings.

Item	Source-reported value	Deployment interpretation
Kind	1.36.1	Reported Kubernetes version for evaluation, not a Kind application version.

Source: Cloud Deployment Administrator and User Guide, PDF pp.4293–4295.

Production release gates:

- Resolve the OpenShift/Kubernetes support-table mismatch with Black Duck. A chart minimum of Kubernetes 1.31 is not a product support statement.
- The reviewed Scan and Storage migration templates enable shell tracing and pass expanded database passwords to Liquibase. Obtain a corrected package or approved mitigation before production installation. Restrict existing migration logs and rotate credentials if exposure is confirmed.
- AI-Assisted Triage fails with multiple Connect replicas in the documented 2026.6 release. Do not approve that combination without a confirmed fix.
- The release notes identify an upgrade failure for bundled PostgreSQL with `onPrem.postgres=true`. Plan database migration separately.

1.3 Validation boundaries

Scan Services configuration was reviewed against supplied templates. Complete parent-chart, triage, and provider-controller behavior must be verified by rendering the actual release and testing the selected environment. Examples explicitly marked “candidate” require that verification. No live deployment, controller installation, or PDF rendering is claimed by this document.

2. Platform architecture

2.1 Full CNC reference architecture

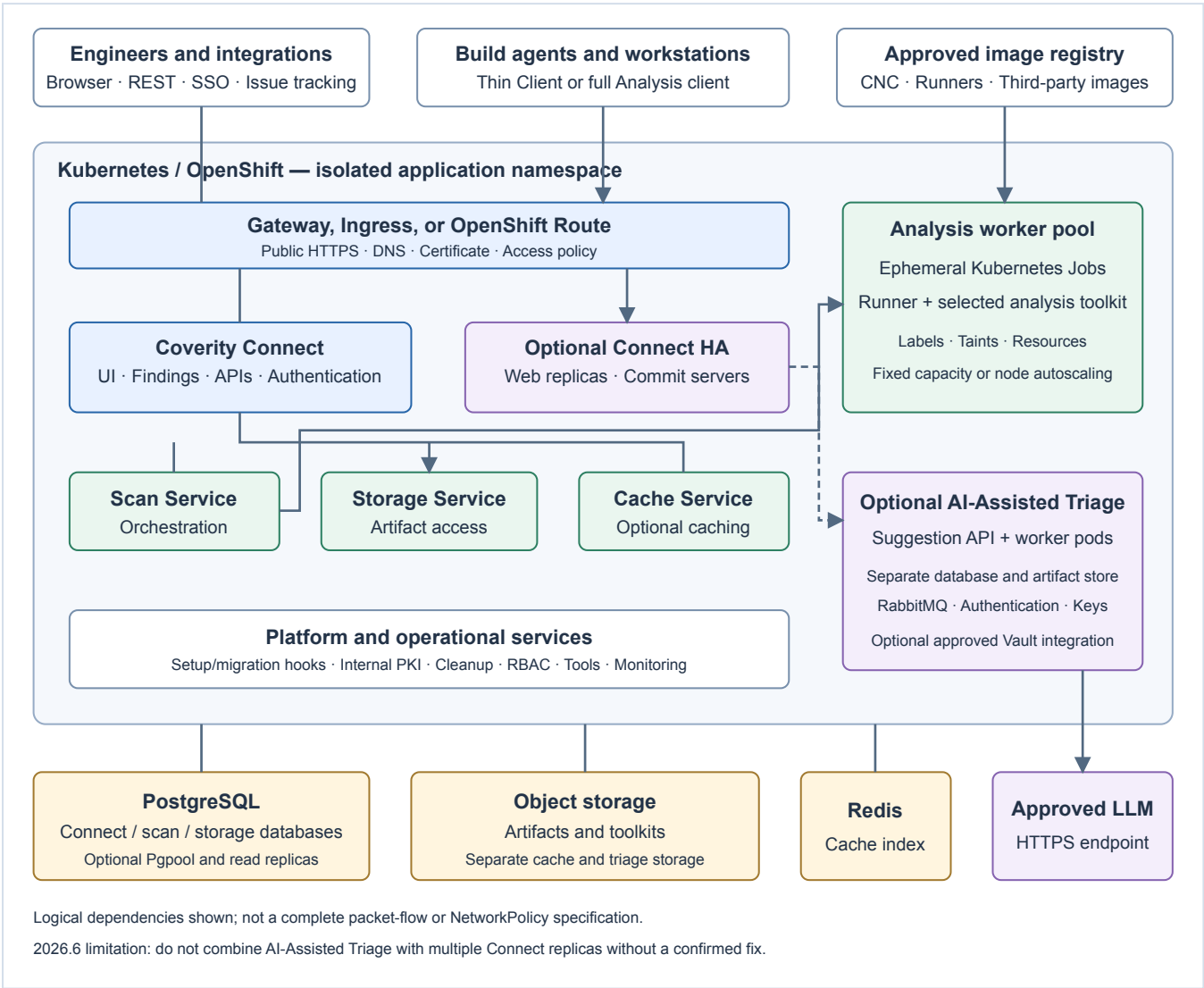


Figure 1. Full platform reference. Data services may be managed externally or deployed in-cluster where supported. Not every optional component is compatible with every other component.

2.2 Connect-only profile

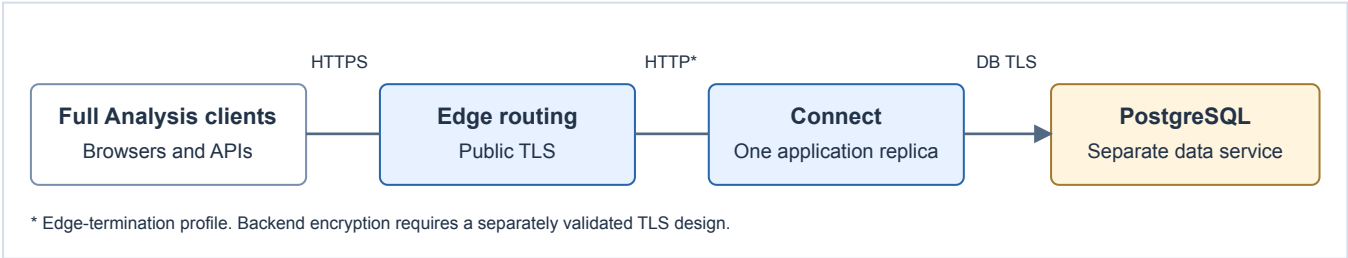


Figure 2. Connect-only has no in-cluster analysis, Redis, or scan artifact bucket. Administrative tools and setup jobs are additional operational workloads.

2.3 Analysis execution flow

1. The client captures the project in its build environment.

2. Connect authenticates the submission; Storage Service provides artifact access.
3. The client uploads capture artifacts to the configured object-storage endpoint.
4. Scan Service schedules an analysis Job on eligible analysis capacity.
5. The Job uses the selected toolkit and optional cache, then commits results to Connect.
6. Connect persists findings in PostgreSQL. Configured logs and analysis artifacts remain available for the retention period.

Both clients and analysis Jobs must reach the actual upload/download endpoints. A working Connect login does not demonstrate a working analysis data path.

3. Sizing and capacity planning

3.1 Separate the resource budgets

Budget	Baseline or sizing method
Each Connect pod	Published minimum: 8 CPUs and 32 GB RAM. Increase for IDIR size, commit concurrency, queue capacity, and user load.
Connect PostgreSQL	Published minimum: 8 CPUs and 32 GB RAM. Approximately 15% of database size is a RAM-planning guideline, subject to the minimum.
Scan/Storage/Cache services	Separate from analysis Jobs. Chart defaults are starting configuration, not production sizing guarantees.
Analysis workers	Size each Job and the number of concurrent Jobs. The largest Job must fit on an individual eligible node.
AI triage	Additional API, worker, database, queue, and artifact-storage budget.
Platform overhead	Operating system, kubelet, CNI, CSI, DaemonSets, controllers, monitoring, rollout surge, and failure reserve.

Pod size is not node size. An 8-CPU, 32-GB node cannot necessarily schedule a pod requesting all 8 CPUs and 32 GB after system reservations. Use allocatable resources, not advertised VM size. Memory figures in source tables use GB; Kubernetes Gi uses binary units.

3.2 Connect commit-sizing reference

The following source measurements were gathered with Coverity 2023.9.0 and an external database of approximately 276 GB and are reproduced in the 2026.6 guide. They are planning evidence, not guaranteed 2026.6 throughput.

IDIR	Commit threads	CPU cores	RAM GB, default queue	RAM GB, maximum queue	Measured commits/hour
2.6 GB	5	8	32	39	110
2.6 GB	10	10	38	45	145
2.6 GB	15	15	80	88	175
2.6 GB	20	20	100	116	239
2.6 GB	25	25	120	128	290
2.6 GB	30	30	144	150	350
12 GB	5	8	50	80	21

IDIR	Commit threads	CPU cores	RAM GB, default queue	RAM GB, maximum queue	Measured commits/hour
12 GB	10	10	80	110	26
12 GB	15	15	100	130	36
12 GB	20	20	120	150	49
12 GB	25	25	130	160	62
12 GB	30	30	144	176	73
36 GB	5	8	56	146	5
36 GB	10	10	120	210	10
36 GB	15	15	160	250	12
36 GB	20	20	200	290	13
36 GB	25	25	220	310	14
36 GB	30	30	264	354	16

Default commit queue capacity is 80 in the cited tests. Increasing queue capacity increases memory demand; it does not increase processing throughput. Apply the full selected allocation to each Connect pod rather than dividing it among replicas.

View loading is particularly sensitive to database CPU. The published view-loading tests used 64 GB of database memory and CPU configurations from 8 to 32 cores. Use representative project issue counts and user concurrency when testing UI latency.

Sources: PDF pp.4004–4015. Validate extracted table values against the original table before contractual capacity commitments.

3.3 Analysis Job sizes

Pool	Job CPU	Job memory	Illustrative node capacity
Small	6.5 cores	26 GB	8 vCPU / 32 GB
Medium	14.5 cores	56 GB	16 vCPU / 64 GB
Large	28.5 cores	108 GB	32 vCPU / 128 GB
Extra large	58.5 cores	222 GB	64 vCPU / 256 GB
Custom	Configured in thousandths of a core	Configured in MB	Must exceed the Job request plus overhead.

CUSTOMNODEPOOL_CPU: 4000 means 4 cores, not 4,000 cores. Confirm the actual resources of a generated Job. Pool flags configure scheduler behavior; they do not provision cloud nodes by themselves.

3.4 Supporting service defaults

Service	CPU request / limit	Memory request / limit	Interpretation
Scan Service	100m / 250m	128Mi / 1512Mi	Supplied chart defaults; the PDF lists a different memory limit.
Storage Service	100m / 250m	128Mi / 512Mi	Increase based on observed load.
Cache Service	500m / 500m	1Gi / 1Gi	Does not include Redis.
Each scan/storage migration Job	250m / 500m	256Mi / 1Gi	Rehearse upgrades against representative data.
Triage API, per pod	500m / 2000m	512Mi / 2Gi	Documented example, not verified against a supplied triage chart.
Triage worker, per pod	1000m / 4000m	2Gi / 8Gi	Multiply by minimum and maximum worker counts.

3.5 Storage planning

For scan artifact storage, the documented estimate is:

Storage GB $\approx$ average retained artifact GB per scan $\times$ scans/day $\times$ retention days + toolkit allowance + log allowance.

The source example uses $0.25 \text{ GB} \times 100 \text{ scans/day} \times 30 \text{ days} + 10 \text{ GB toolkits} + 15 \text{ GB logs} = \mathbf{775 \text{ GB}}$. Treat toolkit and log allowances as workload-dependent. The on-prem guidance separately estimates approximately 8 GB per analysis toolkit version.

- Do not apply provider deletion/expiry rules to the Storage Service bucket.
- Scan retention defaults to 30 days; the documented minimum is 3 days.
- General-purpose cache storage requires its own expiry policy: minimum 8 days, 30 days recommended. S3 Express has separate application-managed TTL behavior.
- Size MinIO raw storage for replication or erasure-coding overhead, usable capacity, growth, and rebuild headroom.
- Database disk planning must include indexes, WAL, maintenance space, backups, and measured growth—not only the current database size.

4. Shared preparation and image supply

4.1 Required decisions and ownership

Decision	Record before installation
Release and platform	Exact versions, architecture, support confirmation.
Topology	Connect-only or scan-enabled; caching; HA; triage; dependency placement.
Networking	Hostnames, public/private access, routing controller, TLS boundaries, egress.
Data services	Endpoints, database names, roles, buckets, retention, backups, owners.
Operations	Deployment owner, DBA, PKI owner, monitoring owner, on-call team.
Acceptance	Representative scan, throughput target, UI latency, restore target, sign-off.

4.2 Prepare a protected working directory

Commands use Bash. Work in a new directory for this deployment. Do not share credentials, rendered Secrets, private keys, or sensitive logs with the guide.

Bash

```
umask 077
mkdir -p charts values secrets review
chmod 700 secrets review

export NS="coverity"
export RELEASE="coverity"
export VERSION="2026.6.0"
export CONNECT_HOST="coverity.example.com"

kubectl config current-context
kubectl version
helm version
kubectl auth can-i --list -n "$NS"
```

Create the namespace if it does not exist. On OpenShift use the approved project creation process or `oc new-project` instead.

Bash

```
kubectl create namespace "$NS"
kubectl get resourcequota,limitrange -n "$NS"
```

4.3 Obtain and preserve the chart

Bash

```
helm pull oci://repo.blackduck.com/cloudnative/cnc \
  --version "$VERSION" --destination charts

helm show chart "charts/cnc-$VERSION.tgz"
tar -xzf "charts/cnc-$VERSION.tgz" -C charts
export CHART="$PWD/charts/cnc"

helm dependency list "$CHART"
helm show values "$CHART" > review/chart-defaults.yaml

# macOS; on Linux use sha256sum.
shasum -a 256 "charts/cnc-$VERSION.tgz" > review/chart-sha256.txt
```

Do not run dependency update/build as a routine repair. If packaged dependencies are missing or inconsistent, obtain a corrected release package.

4.4 Image inventory

Profile	Black Duck image names
Connect	cim-web, cim-tools, cim-downloads
Scan Services	scan-service, scan-service-migration, storage-service, storage-service-migration, common-infra, job-runner
Caching	cache-service
AI triage	triage-suggestion-service-api, triage-suggestion-service-worker

Black Duck application images use `repo.blackduck.com/containers`. For the reviewed Connect/Scan image helpers, OpenShift uses the `2026.6.0-ubi` tag; other environments use `2026.6.0`. Verify triage image platform/tag handling separately. Third-party images do not automatically share these names or tag conventions.

- Mirror all enabled application, init, hook, sidecar, and dynamically launched Job images, including dependencies and their utility images.
- Image-pull credentials, toolkit synchronization credentials, and registry CA trust are separate requirements.
- Cloud registry tokens may expire. Prefer supported node identity integration or a maintained credential rotation mechanism.
- A legacy Bitnami override restores image availability, not security maintenance. Approve image provenance, vulnerabilities, licensing, and update ownership.

5. Database, secrets, and certificate trust

5.1 Database contract

Use explicit logical database names: `cim`, `scan`, and `storage`; add a separate triage database when needed. They may share an appropriately sized supported PostgreSQL server, but capacity, permissions, and recovery requirements must cover all workloads.

1. Provision the supported PostgreSQL engine with private network access.
2. Have the DBA create the required databases, login roles, ownership, encoding, collation, and release-required extensions.
3. Grant schema initialization/upgrade permissions to the identities used by the setup and migration Jobs. Do not grant superuser solely for convenience.
4. Require TLS and distribute the correct CA bundle.
5. Verify database authentication and TLS from the workload network.
6. Configure backups, recovery-point and recovery-time targets, and restore testing.

For Azure Flexible Server, allow the required extensions such as `uuid-oss` and `btree_gist` in the server extension list. Preserve existing allowed extensions when updating it. Do not disable `require_secure_transport` as an installation shortcut.

5.2 Secret inventory

Example Secret	Required keys / contents
<code>cnc-registry</code>	<code>.dockerconfigjson</code> ; type <code>kubernetes.io/dockerconfigjson</code> .
<code>cnc-connect-db</code> , <code>cnc-scan-db</code> , <code>cnc-storage-db</code>	<code>host</code> , <code>port</code> , <code>username</code> , <code>password</code> .
<code>cnc-connect-license</code> , <code>cnc-analysis-license</code>	<code>license.dat</code> ; correct license entitlement for each use.
<code>cnc-admin</code>	<code>password</code> ; initial setup or intentional password reset.
<code>cnc-public-tls</code>	<code>tls.crt</code> , <code>tls.key</code> .
<code>cnc-redis-auth</code>	password for Cache Service.
<code>cnc-redis-ca</code>	<code>ca.crt</code> .
<code>cnc-tool-sync</code>	<code>username</code> , <code>password</code> for Black Duck toolkit downloads.
AWS static-key alternative	<code>aws_access_key</code> , <code>aws_secret_key</code> . Not required for the reviewed IRSA path.
GCS credential Secret	<code>key.json</code> for Cache Service; Storage Service has a configurable key name.
Azure Storage Service	Entra client-secret mode: <code>azure_endpoint</code> , <code>azure_tenant_id</code> , <code>azure_client_id</code> , <code>azure_client_secret</code> .
Azure Cache Service	Previous four keys plus <code>azure_subscription_id</code> and <code>azure_resource_group</code> .
MinIO credential Secret	<code>root-user</code> , <code>root-password</code> for the native MinIO adapter.

Prepare protected files for passwords and credentials; do not put literal secrets in shell history. The following example creates the Connect database Secret. Repeat with the appropriate endpoint, identity, and Secret name for other databases.

Bash

```
export DB_HOST="REPLACE_DATABASE_HOST"
export DB_PORT="5432"
export DB_USER="REPLACE_DATABASE_USER"

kubectl create secret generic cnc-connect-db -n "$NS" \
  --from-literal=host="$DB_HOST" \
  --from-literal=port="$DB_PORT" \
  --from-literal=username="$DB_USER" \
  --from-file=password=secrets/connect-db-password

kubectl create secret generic cnc-registry -n "$NS" \
  --type=kubernetes.io/dockerconfigjson \
  --from-file=.dockerconfigjson=secrets/registry-config.json

kubectl create secret generic cnc-connect-license -n "$NS" \
  --from-file=license.dat=secrets/connect-license.dat

kubectl create secret generic cnc-analysis-license -n "$NS" \
  --from-file=license.dat=secrets/analysis-license.dat

kubectl create secret generic cnc-admin -n "$NS" \
  --from-file=password=secrets/admin-password

kubectl create secret tls cnc-public-tls -n "$NS" \
  --cert=secrets/public-chain.pem \
  --key=secrets/public-key.pem
```

For Connect-only, omit the Analysis license unless it is required for a separate client distribution arrangement. Administrator passwords must meet the documented 8–32 character and character-class requirements.

5.3 Trust and encryption material

Bash

```
kubectl create configmap cnc-trust -n "$NS" \
  --from-file=postgres-root.pem=secrets/postgres-root.pem

# Caching only:
kubectl create secret generic cnc-redis-auth -n "$NS" \
  --from-file=password=secrets/redis-password

kubectl create secret generic cnc-redis-ca -n "$NS" \
  --from-file=ca.crt=secrets/redis-ca.pem
```

- Keep the database bundle under the key `postgres-root.pem`. For AWS, the supplied guide also requests individual PEM certificates alongside the original RDS bundle for truststore import.
- Add approved LDAP, SMTP, Jira, storage, or proxy CA certificates as required. Private keys do not belong in a ConfigMap.
- Use `verify-full` when the supported database/driver combination and certificate hostname permit it. `verify-ca` validates the CA but does not verify the server hostname.
- Public edge certificates, internal service certificates, database trust, and triage encryption keys have different purposes and renewal procedures.

- For migration, preserve the existing `cm.ldap.key`. For a new installation, generate and securely retain a unique key.

Bash: new installation only; refuse to overwrite an existing key

```
(  
  set -o noclobber  
  openssl rand -base64 32 | tr -d '\n' > secrets/ldap-key  
)  
chmod 600 secrets/ldap-key
```

Kubernetes Secrets require RBAC, encryption-at-rest configuration, and backup protection. Base64 encoding is not encryption.

6. Shared Helm configuration

The examples use four layers: **base** → **profile** → **provider/routing** → **site overrides**. Values from later files take precedence. Lists are generally replaced rather than merged element by element. YAML does not expand shell variables.

6.1 Base configuration

Create `values/base.yaml`. The Connect allocation is a minimum-based starting point, not a recommendation for every production workload. Increase it according to section 3 before installation.

YAML: `values/base.yaml`

```
global:
  imageRegistry: "repo.blackduck.com/containers"
  imageVersion: "2026.6.0"
  imageTagSuffix: ""
  imagePullSecret: "cnc-registry"
  licenseSecretName: "cnc-connect-license"
  keygen:
    enabled: false
  postgres:
    database: "postgres"
    existingSecret: "cnc-connect-db"
    sslmode: "verify-ca"
  trust-stores:
    enabled: true
    configmapName: "cnc-trust"
  ingress:
    enabled: false
    hosts:
      - "REPLACE_CONNECT_HOST"
    tls:
      - secretName: "cnc-public-tls"
        hosts:
          - "REPLACE_CONNECT_HOST"
  proxy:
    enabled: false

onPrem:
  postgres: false
  redis: false
  minio: false

scan-services:
  enabled: false

triage-suggestion-service:
  enabled: false

cnc-db-admin:
  enabled: false

cim:
  automountServiceAccountToken: false
  podSecurityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  postgres:
    database: "cim"
    existingSecret: "cnc-connect-db"
    sslmode: "verify-ca"
```

```

cimweb:
  enabled: true
  replicas: 1
  webUrl: "https://REPLACE_CONNECT_HOST"
  contextPath: ""
  exposeCommitPort: false
  exposeMetrics: true
  extraProperties:
    connect.enable.metrics: true
  adminPasswordSecret: "cnc-admin"
  loadBalancer:
    trustedRegex: "REPLACE_TRUSTED_PROXY_REGEX"
  updateLicense:
    enabled: true
    force: false
  tlsSidecar:
    enabled: false
  resources:
    requests:
      cpu: "8"
      memory: "32Gi"
    limits:
      cpu: "8"
      memory: "32Gi"
cimtools:
  enabled: true
  automountServiceAccountToken: false
setupJob:
  enabled: true
  createDatabases: false
  activeDeadlineSeconds: 3600
commit-server:
  replicas: 0
pgpool:
  enabled: false
ingress:
  enabled: false
route:
  enabled: false
gateway:
  enabled: false
  create: false

minioGateway:
  gateway:
    enabled: false

```

Keep ingress hostname and TLS metadata consistent even when no Ingress object is created: the reviewed Scan Services helper uses these fields to form HTTPS web origins. Confirm the rendered Connect and commit URLs.

Restrict the trusted proxy expression to the actual approved proxy sources. Do not leave a catch-all expression. Metrics must remain private.

6.2 Scan Services profile

Create `values/scan.yaml` for in-cluster analysis. Caching is explicitly disabled until a provider cache configuration is added.

YAML: `values/scan.yaml`

```

scan-services:
  enabled: true
  mode: "CNC"

```

```

ingress:
  enabled: false
postgres:
  database: "postgres"
cache-service:
  enabled: false
scan-service:
  licenseSecretName: "cnc-analysis-license"
  automountServiceAccountToken: false
  postgres:
    database: "scan"
    existingSecret: "cnc-scan-db"
    createDatabase: false
    sslmode: "verify-ca"
  migrateJob:
    enabled: true
  environment:
    COVANALYSIS_DEFAULTPOOLTYPE: "small"
    SMALLNODEPOOL_ENABLE: true
    MEDIUMNODEPOOL_ENABLE: false
    LARGENODEPOOL_ENABLE: false
    EXTRALARGENODEPOOL_ENABLE: false
    MULTIPLEJOBSPERNODE_ENABLE: false
  tolerations:
    - key: "NodeType"
      operator: "Equal"
      value: "ScannerNode"
      effect: "NoSchedule"
  retention:
    enabled: true
    minutes: 43200
    cleanupInterval: "30m"
  jobRunner:
    uploadArtifacts: "All"
  tools:
    sync:
      enabled: false
  observability:
    exposeMetrics: true
storage-service:
  automountServiceAccountToken: false
  postgres:
    database: "storage"
    existingSecret: "cnc-storage-db"
    createDatabase: false
    sslmode: "verify-ca"
  migrateJob:
    enabled: true
  observability:
    exposeMetrics: true
common-infra:
  automountServiceAccountToken: false

```

Security and scheduling: the supplied templates explicitly project Kubernetes API tokens for Scan Service and common infrastructure when automatic mounting is false. Do not replace them with a Connect application token. Scan-service tolerations are also used by generated analysis Jobs; pin the long-running services to application nodes when required to prevent them using analysis capacity.

The parent chart must provide Connect integration, internal TLS configuration, and the referenced Connect ConfigMap. Render the complete CNC chart—not the Scan Services subchart in isolation—as the installation validation target.

6.3 Site overrides

Use `values/site.yaml` for reviewed resource allocations, node selectors, proxy trust, custom domains, and other site-specific requirements. An empty file containing `{}` is acceptable only if all necessary settings are already present in the selected files.

For a migration or later upgrade, set `cim.cimweb.adminPasswordSecret: ""` unless an intentional password reset is required. Retain all other site-specific values.

7. AWS EKS

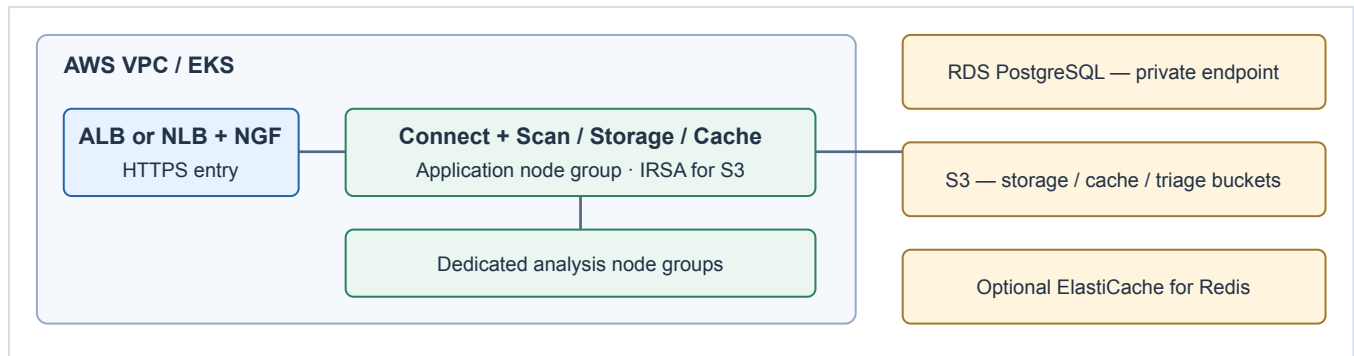


Figure 3. AWS reference. ALB and NGF are alternative routing paths. AI triage adds the components in section 15.

7.1 Infrastructure preparation

1. Select a supported EKS version and region. Create or use approved VPC, subnets, endpoint access, DNS, and security groups.
2. Provision application node groups and dedicated analysis node groups. Use the sizes and scheduling rules in sections 3 and 13.
3. Provision RDS PostgreSQL privately and complete the database contract. Do not infer Aurora support from PostgreSQL compatibility; the supplied database chapter contains conflicting RDS/Aurora instructions requiring confirmation.
4. Create private S3 storage and cache buckets. Enable public-access blocking and approved encryption. Apply expiry only to the cache bucket.
5. Create separate IRSA identities for Storage and Cache Services where least-privilege separation is required.
6. For caching, provision a supported Redis deployment and TLS credentials.
7. Provision the selected routing controller and certificates before CNC installation.

Bash: connect to an existing EKS cluster

```
export AWS_REGION="REPLACE_REGION"
export CLUSTER_NAME="REPLACE_EKS_CLUSTER"

aws sts get-caller-identity
aws eks describe-cluster --name "$CLUSTER_NAME" --region "$AWS_REGION"
aws eks update-kubeconfig --name "$CLUSTER_NAME" --region "$AWS_REGION"
kubectl get nodes -L kubernetes.io/arch,pool-type
```

7.2 S3 identity and permissions

IRSA requires an EKS OIDC provider and an IAM role trust policy restricted to the intended Kubernetes namespace and service account. The cluster IAM role, node IAM role, load-balancer-controller role, and application S3 roles are distinct.

The documented S3 actions include bucket CORS read/write, bucket listing, object read/write/delete, multipart-abort and part-listing operations. Cache configuration also requires lifecycle inspection and, where used, modification. Scope bucket actions to bucket ARNs and object actions to object ARNs. Include KMS permissions if the selected encryption design requires them.

Bash: after the reviewed policies and OIDC provider exist

```
eksctl create iamserviceaccount \
  --cluster "$CLUSTER_NAME" --region "$AWS_REGION" \
  --namespace "$NS" --name cnc-storage \
  --attach-policy-arn "REPLACE_STORAGE_POLICY_ARN" --approve

eksctl create iamserviceaccount \
  --cluster "$CLUSTER_NAME" --region "$AWS_REGION" \
  --namespace "$NS" --name cnc-cache \
  --attach-policy-arn "REPLACE_CACHE_POLICY_ARN" --approve
```

Verify the role annotation, token injection, and application access after admission. Do not assume EKS Pod Identity is interchangeable with this chart's documented IRSA path without testing the application SDKs.

7.3 S3 and cache values

YAML: values/aws-storage.yaml — scan-enabled profile with caching

```
scan-services:
  storage-service:
    storageType: "s3"
    s3:
      bucket: "REPLACE_STORAGE_BUCKET"
      region: "REPLACE_REGION"
      serviceAccount: "cnc-storage"
  cache-service:
    enabled: true
    storageProvider: "aws"
    bucketName: "REPLACE_CACHE_BUCKET"
    aws:
      region: "REPLACE_REGION"
      serviceAccount: "cnc-cache"
      s3Express:
        enabled: false
    redis:
      host: "REPLACE_REDIS_HOST"
      port: 6379
      database: "1"
      authEnabled: true
      passwordSecret: "cnc-redis-auth"
      secure: true
      cacertSecret: "cnc-redis-ca"
      verifyHostName: true
```

For no caching, set `cache-service.enabled: false` and omit the cache bucket, Redis, and cache identity configuration.

ElastiCache restrictions in the supplied guide: no serverless caching, cluster mode disabled, zero replicas, and `maxmemory-policy=noeviction`. The suggested 1 GB Redis starting size is not an availability or capacity guarantee. Review these limitations before committing to a cache HA requirement.

7.4 Routing choices

- **NGF behind NLB:** use the shared NGF routing values in section 12. Pin the NGF release and configure the generated data-plane Service using the mechanism supported by that NGF release.

- **Native ALB:** use a compatible AWS Load Balancer Controller, its CRDs, an accepted GatewayClass, and reviewed LoadBalancerConfiguration / TargetGroupConfiguration resources. Confirm certificate handling for the selected controller version; do not assume a Kubernetes TLS Secret automatically becomes an ALB certificate. ACM configuration is commonly required.
- Discover the actual load-balancer DNS name. Use a CNAME or an appropriate Route 53 Alias record. Do not hardcode an ephemeral load-balancer IP.
- Validate source-IP preservation before enabling IP allowlists. NLB target-group attributes, target type, and proxy behavior matter; one Service setting is not proof.

7.5 S3 Express variation

Use `storageType: "s3Express"` and the `storage-service.s3Express` block instead of `s3`. For Cache Service set `aws.s3Express.enabled: true` and an approved `ttlDays`. Review directory-bucket region/AZ availability and S3 Express-specific IAM actions against current AWS documentation.

The 2026.6 guide identifies a restriction on uploading `coverity-all-platforms-<version>.tar.gz` through Connect with S3 Express. Validate the alternate toolkit-provisioning path before selecting it.

7.6 AWS acceptance additions

Verify RDS CA trust; S3 CORS; IRSA credentials; browser and CI access to presigned URLs; cache lifecycle; Redis TLS; analysis-node scaling; and load-balancer health checks. Check private endpoint policies as well as security groups.

Sources: PDF pp.3853–3875, 3915–3917, 3976–3987.

8. Microsoft Azure AKS

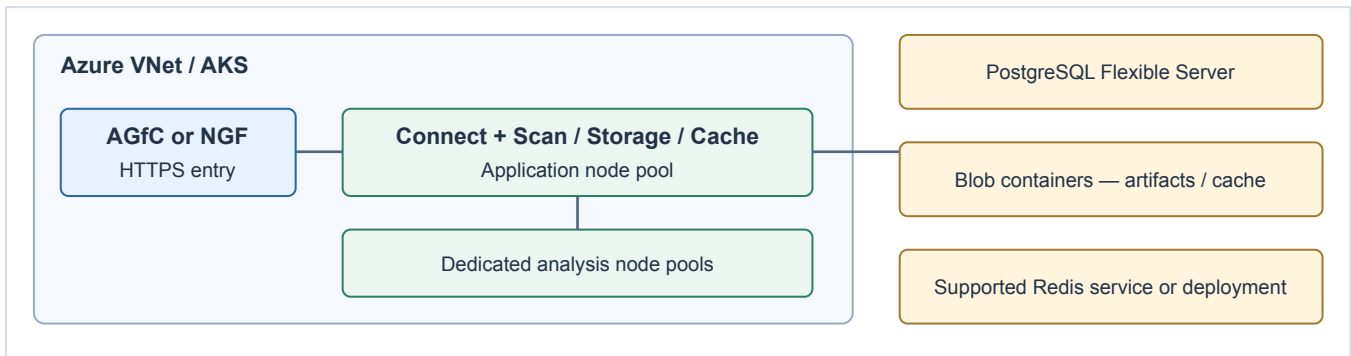


Figure 4. Azure reference. Controller workload identity and application storage credentials are separate integrations.

8.1 Infrastructure preparation

1. Select supported AKS, network-plugin, region, and routing-controller versions.
2. Provision private connectivity and DNS for PostgreSQL, storage, and Redis.
3. Provision Flexible Server and preserve TLS enforcement. Allow the required extensions without replacing unrelated extension entries.
4. Create separate private storage and cache containers. Scope the storage-account lifecycle rule to the cache container only.
5. Configure Entra application credentials and the necessary data-plane and management-plane permissions for storage/CORS/cache-lifecycle operations.
6. Provision dedicated analysis pools and the chosen edge controller.

Bash: connect to an existing AKS cluster

```
export AZURE_SUBSCRIPTION="REPLACE_SUBSCRIPTION_ID"
export RESOURCE_GROUP="REPLACE_RESOURCE_GROUP"
export CLUSTER_NAME="REPLACE_AKS_CLUSTER"

az account set --subscription "$AZURE_SUBSCRIPTION"
az aks show -g "$RESOURCE_GROUP" -n "$CLUSTER_NAME"
az aks get-credentials -g "$RESOURCE_GROUP" -n "$CLUSTER_NAME"
kubectl get nodes -L kubernetes.io/arch,pool-type
```

8.2 Storage authentication

The reviewed Storage Service supports `aadClientSecret` and `sharedKey`. Use the documented Entra client-secret path rather than the discouraged shared-key default. Do not silently replace it with AKS Workload Identity: the supplied Cache Service template expects explicit credential keys.

Use appropriately scoped roles. Subscription-wide Contributor is not a normal application runtime requirement. Blob data access alone may not cover the management operations used for CORS or lifecycle inspection.

```
scan-services:
  storage-service:
    storageType: "azure"
    azure:
      container: "REPLACE_STORAGE_CONTAINER"
      storageAccountName: "REPLACE_STORAGE_ACCOUNT"
      authMode: "aadClientSecret"
      secret:
        name: "cnc-azure-storage"
  cache-service:
    enabled: true
    storageProvider: "azure"
    bucketName: "REPLACE_CACHE_CONTAINER"
    azure:
      secret: "cnc-azure-cache"
  redis:
    host: "REPLACE_REDIS_HOST"
    port: 6380
    database: "1"
    authEnabled: true
    passwordSecret: "cnc-redis-auth"
    secure: true
    cacertSecret: "cnc-redis-ca"
    verifyHostName: true
```

The Redis port above is illustrative. Use the actual service TLS port and confirm logical database 1 is supported. Do not assume every Azure-managed Redis SKU, clustered mode, or replacement service is supported by this release.

8.3 Routing choices

- **NGF:** use section 12 and configure the generated Service's internal/public exposure through the selected controller's supported mechanism.
- **Application Gateway for Containers:** validate Azure CNI compatibility, controller identity, delegated subnet, role assignments, GatewayClass, and ApplicationLoadBalancer association before installing CNC.
- Do not reproduce preview-registration commands or cluster network migrations without checking the current supported Azure procedure.
- Use the controller-reported address type to select DNS records. An FQDN is not an A-record value; apex alias support depends on the DNS provider.
- NGF snippets and client settings are not portable Azure ALB policy resources.

8.4 Azure acceptance additions

Test extension availability, PostgreSQL TLS, Entra credential expiry/rotation, Blob CORS and signed URLs, cache-container-only lifecycle scope, private DNS, Redis authentication, and controller backend health.

Sources: PDF pp.3888–3903, 3987–3998, 4452–4455.

9. Google Kubernetes Engine

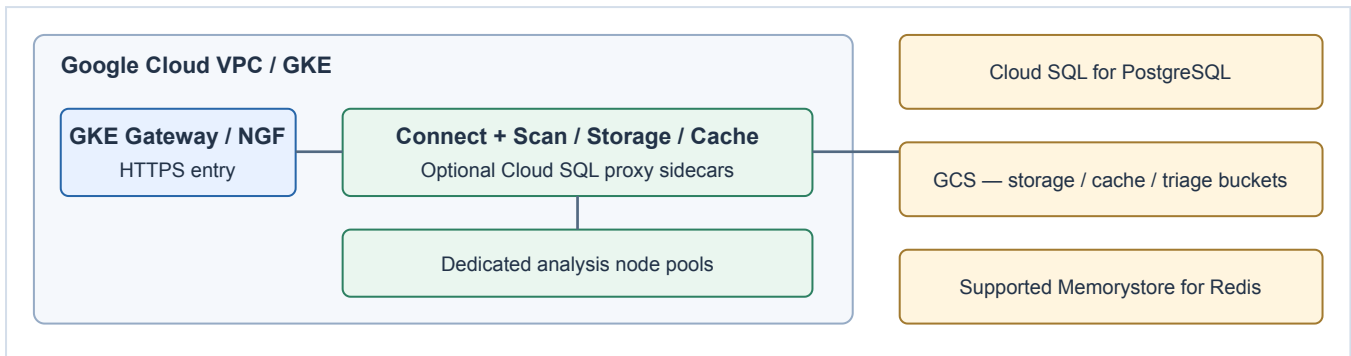


Figure 5. GKE reference. Cloud SQL connectivity, object-storage authentication, and node image-pull identity are distinct paths.

9.1 Infrastructure preparation

1. Use a supported GKE configuration with enough control over nodes and scheduling. Do not assume Autopilot equivalence to a Standard cluster without confirmation.
2. Provision application and analysis pools with private connectivity to data services.
3. Provision Cloud SQL PostgreSQL and approved TLS or Cloud SQL Auth Proxy connectivity.
4. Create private regional GCS storage and cache buckets near the workloads. Avoid copying a multi-region example into a latency-sensitive cache design.
5. Grant the runtime identities object access and required bucket-management permissions for CORS/lifecycle operations.
6. Provision a supported Redis service with authentication, TLS, and noeviction.

Bash: connect to an existing regional GKE cluster

```
export GCP_PROJECT="REPLACE_PROJECT"
export GCP_REGION="REPLACE_REGION"
export CLUSTER_NAME="REPLACE_GKE_CLUSTER"

gcloud container clusters get-credentials "$CLUSTER_NAME" \
  --project "$GCP_PROJECT" --region "$GCP_REGION"

kubectl get nodes -L kubernetes.io/arch,pool-type
kubectl get gatewayclass
```

9.2 GCS and cache values

The documented key-file path is shown because the supplied Cache Service template explicitly mounts `key.json`. Organizations prohibiting service-account keys need a separately validated workload-identity integration; simply omitting the Secret does not satisfy this template.

```
scan-services:
  storage-service:
    storageType: "gcs"
    gcs:
      bucket: "REPLACE_STORAGE_BUCKET"
      secret:
        name: "cnc-gcs-storage"
        key: "key.json"
  cache-service:
    enabled: true
    storageProvider: "gcp"
    bucketName: "REPLACE_CACHE_BUCKET"
    gcp:
      project: "REPLACE_PROJECT"
      secret: "cnc-gcs-cache"
  redis:
    host: "REPLACE_REDIS_HOST"
    port: 6378
    database: "1"
    authEnabled: true
    passwordSecret: "cnc-redis-auth"
    secure: true
    cacertSecret: "cnc-redis-ca"
    verifyHostName: true
```

Use the actual Redis TLS endpoint and port. Verify certificate hostname behavior for the chosen service rather than disabling verification to make a test pass.

9.3 Cloud SQL connectivity

- **Direct private connection:** configure database Secrets with the private endpoint, approved CA, and supported TLS mode.
- **Auth Proxy:** use a pinned proxy image, proper IAM identity, private-IP configuration where required, explicit arguments, resources, and readiness behavior.
- Proxy sidecars must be present in every relevant application and setup/migration pod. The reviewed Scan Services templates support separate workload and Job sidecar lists.
- For proxy connections, database Secret host values must match the local listener. A global host setting will not override a host read from an existing Secret.
- Do not blindly inherit direct-database TLS settings on the localhost leg. Document separately how the application-to-proxy and proxy-to-Cloud-SQL legs are protected.

9.4 Native GKE routing

Enable Gateway API through GKE's supported cluster procedure and use an available accepted GatewayClass. Examples include gke-l7-global-external-managed and gke-l7-ilb; class availability and features depend on the cluster and region.

YAML: values/gcp-routing.yaml — candidate parent-chart routing configuration

```
cim:
  gateway:
    enabled: true
    create: true
    name: "coverity-gateway"
    gatewayClassName: "gke-l7-global-external-managed"
    hostnames:
      - "REPLACE_CONNECT_HOST"
    listeners:
      http:
        enabled: true
        port: 80
        redirect: true
      https:
        enabled: true
        port: 443
        tlsSecretName: "cnc-public-tls"
    healthCheck:
      requestPath: "/login/login.htm"
      commitServerRequestPath: "/login/login.htm"
```

Verify HealthCheckPolicy attachment and NEG/backend readiness. Use GCPBackendPolicy for supported Gateway backend settings such as Cloud Armor attachment; do not copy legacy Ingress BackendConfig annotations into a Gateway design and assume they apply.

NGF-specific allowlist and body-size fields do not configure the native GKE load balancer. Discover its address from the Gateway status.

Sources: PDF pp.3875–3887, 3964–3976, 4080–4086. Provider-native policy details require current Google documentation validation.

10. Red Hat OpenShift

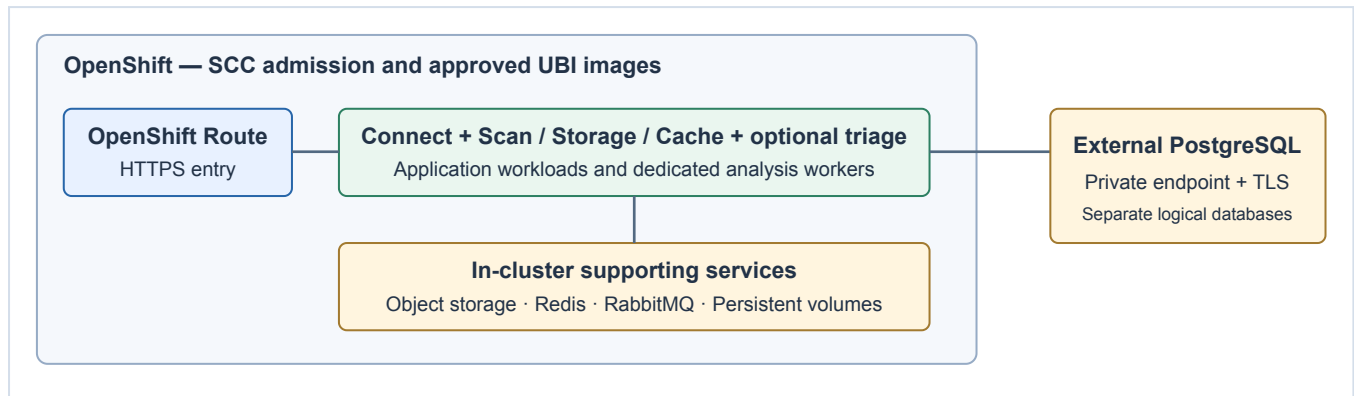


Figure 6. OpenShift variant with external PostgreSQL and other dependencies hosted in-cluster. Operators and Helm releases may have separate ownership.

10.1 Platform requirements

- Resolve the release support combination before selecting an OpenShift version.
- Use the documented UBI application image variants and verify architecture availability.
- Allow SCC admission to assign permitted IDs. Avoid fixed UID overrides unless required and approved for a specific dependency.
- Validate security settings for application pods, migration hooks, certificate Jobs, analysis Jobs, and third-party StatefulSets—not only Connect.
- The reviewed migration templates set `-Dcom.redhat.fips=false` on OpenShift. Do not claim FIPS compliance without a separate vendor-supported review.

Bash

```
oc whoami
oc version
oc api-versions | grep route.openshift.io
oc get resourcequota,limitrange -n "$NS"

oc get pods -n "$NS" \
  -o 'custom-columns=NAME:.metadata.name,SCC:.metadata.annotations.openshift\.io/scc'
```

10.2 Platform values

YAML: values/openshift-platform.yaml

```
global:
  imageTagSuffix: "-ubi"

cim:
  gateway:
    enabled: false
    create: false
  ingress:
    enabled: false
  route:
    enabled: false

scan-services:
  ingress:
    enabled: false
```

This profile manages the public Route separately. It avoids dependency on a Helm render-time certificate lookup. The complete rendered application service must expose the intended backend port.

10.3 Create the Connect Route

Bash: after Connect is ready

```
oc create route edge "$RELEASE-public" \
  -n "$NS" \
  --service="$RELEASE-cim" \
  --port=p8080 \
  --hostname="$CONNECT_HOST" \
  --cert=secrets/public-chain.pem \
  --key=secrets/public-key.pem \
  --insecure-policy=Redirect

oc annotate route "$RELEASE-public" -n "$NS" \
  haproxy.router.openshift.io/timeout=600s --overwrite

oc get route "$RELEASE-public" -n "$NS"
oc get route "$RELEASE-public" -n "$NS" \
  -o 'jsonpath={.status.ingress[*].conditions}{"\n"}'
```

Edge termination forwards HTTP to Connect. Re-encrypt requires an approved backend TLS listener, destination CA, and any required client-authentication configuration; changing only the Route termination field is insufficient.

10.4 In-cluster storage, Redis, and RabbitMQ

Use approved operators or reviewed Helm releases. Apply the dependency contract in section 11. Provide stable Service DNS names, persistent volumes where required, resource sizing, credentials, TLS, and backup ownership before enabling consumers.

For S3-compatible storage, use the adapter matching the documented backend. The S3-compatible profile below is a template-reviewed candidate; confirm presigned URLs, region, path style, and CA handling against the actual backend.

YAML: values/openshift-storage.yaml

```
scan-services:
  storage-service:
    storageType: "s3"
    s3:
      bucket: "REPLACE_STORAGE_BUCKET"
      region: "REPLACE_STORAGE_REGION"
      secret:
        name: "cnc-object-storage"
        serviceAccount: ""
    endpoint:
      internal:
        url: "https://REPLACE_INTERNAL_STORAGE_ENDPOINT"
      external:
        url: "https://REPLACE_CLIENT_STORAGE_ENDPOINT"
  cache-service:
    enabled: true
    storageProvider: "aws"
    bucketName: "REPLACE_CACHE_BUCKET"
    aws:
      region: "REPLACE_STORAGE_REGION"
      secret: "cnc-object-storage"
      serviceAccount: ""
      s3Express:
        enabled: false
    environment:
      AWS_ENDPOINT: "https://REPLACE_INTERNAL_STORAGE_ENDPOINT"
      CUSTOM_CACERT_ENABLED: true
      CUSTOM_CACERT_PATH: "/tmp/storage-certs"
      CERT_STORE_PATH: "/tmp/storage-truststore.jks"
      CERT_STORE_TYPE: "JKS"
    extraVolumes:
      - name: storage-ca
        configMap:
          name: cnc-storage-ca
    extraVolumeMounts:
      - name: storage-ca
        mountPath: /tmp/storage-certs
        readOnly: true
    redis:
      host: "REPLACE_REDIS_SERVICE_DNS"
      port: 6379
      database: "1"
      authEnabled: true
      passwordSecret: "cnc-redis-auth"
      secure: true
      cacertSecret: "cnc-redis-ca"
      verifyHostName: true
```

Create cnc-storage-ca with the approved storage CA. Add that CA to cnc-trust for Scan/Storage access. Where required, configure Scan Service's custom CA path using TLS_CUSTOM_ENABLED and TLS_CUSTOM_CERT_PATH.

For nonstandard storage domains, configure `cim.cimweb.extraProperties.storage.service.custom.domains` as a mapping entry with approved domain names. Validate browser CSP and storage CORS separately. Do not expose the MinIO administrative console as the artifact endpoint.

10.5 Acceptance additions

Check Route admission, router-to-pod policy, SCCs on generated analysis Jobs, storage endpoint reachability from browsers and CI, signed URL behavior, PVC binding, Redis credentials, and object-storage persistence after pod replacement.

11. Customer-managed Kubernetes and dependencies

Use this path for a confirmed supported Kubernetes environment where the customer owns infrastructure and data services. “Runs on Kubernetes” is not sufficient to establish product support for an unlisted distribution.

11.1 Infrastructure contract

Service	Required delivery	Verification
Cluster	Supported version, HA control plane where required, CNI, DNS, time synchronization, node lifecycle and security policy.	Node health, DNS, API reachability and workload admission.
StorageClasses / CSI	Appropriate access modes, performance, expansion, topology and reclaim policy.	PVC binding, restart persistence, backup and restoration.
PostgreSQL	Supported engine, separate database identities, TLS, capacity, backups.	Authentication, schema jobs, restore rehearsal.
Object storage	Supported S3-compatible or native backend, storage/cache/triage buckets, internal and client endpoints.	Upload, download, multipart, CORS, signed URLs, lifecycle isolation.
Redis	Supported nonclustered configuration where required, password, TLS, logical database, noeviction.	Cache startup and representative cache reuse.
RabbitMQ	Triage-only broker, matching credentials/vhost/queue, durability, TLS design, persistent storage.	Publish/consume, restart recovery, backlog monitoring.
Routing	Approved controller, compatible CRDs, DNS and PKI.	HTTPS, redirects, backend health and upload sizes.

11.2 Bundled dependencies versus separate releases

The CNC package includes optional dependency integrations. Their enable flags are not a complete production configuration.

YAML: documented dependency switches — not a complete installation values file

```
onPrem:
  postgres: false
  redis: true
  minio: true

triage-suggestion-service:
  rabbitmq:
    enabled: true
```

Before selecting this pattern, complete the packaged dependency values for credentials, persistence, storage classes, TLS, resources, images, and security contexts. Verify hook ordering against fresh dependencies: pre-install migration Jobs cannot depend on an unavailable database.

- Separate dependency releases or operators simplify independent upgrades, backup ownership, and recovery. They may still run in the same cluster.
- `onPrem.postgres: false` means the CNC release does not provision PostgreSQL; it does not necessarily mean the database is outside Kubernetes.

- Cache Service requires a Secret key named password; bundled Redis commonly uses redis-password. Explicitly reconcile the two.
- MinIO native adapter credentials use root-user and root-password. Verify that the selected dependency produces those keys.
- Bucket creation and cache lifecycle setup must finish before Cache Service is expected to become ready.
- Do not assume setting an image registry prefix works identically for Black Duck images and Bitnami subcharts. Verify each complete rendered image reference.

11.3 Native MinIO adapter alternative

YAML: provider adapter example; combine with the shared scan profile and Redis settings

```
scan-services:
  storage-service:
    storageType: "minio"
    minio:
      bucket: "REPLACE_STORAGE_BUCKET"
      region: "REPLACE_REGION"
      secret:
        name: "cnc-minio-auth"
    endpoint:
      internal:
        url: "https://REPLACE_INTERNAL_MINIO_ENDPOINT"
      external:
        url: "https://REPLACE_CLIENT_MINIO_ENDPOINT"
  cache-service:
    enabled: true
    storageProvider: "minio"
    bucketName: "REPLACE_CACHE_BUCKET"
    minio:
      host: "REPLACE_MINIO_HOST"
      port: 9000
      secret: "cnc-minio-auth"
      secure: true
      verifyHostName: true
      cacert: "REPLACE_CA_VALUE_AS_REQUIRED_BY_ADAPTER"
```

The supplied template passes the cacert value directly to the application. Confirm whether the image expects certificate content or a mounted path before finalizing this adapter configuration.

12. Routing, TLS, and network access

12.1 Controller selection

Environment	Routing options	Important distinction
AWS	NGF behind NLB; compatible native ALB Gateway or Ingress.	Controller IAM and ACM are separate from application IRSA.
Azure	NGF; compatible Application Gateway for Containers.	Verify networking mode, identity and subnet association.
GCP	NGF; native GKE Gateway.	Gateway policies differ from legacy Ingress BackendConfig.
OpenShift	Native Route; other controllers only with approval.	Route termination must match backend protocol and authentication.
Customer-managed Kubernetes	Reviewed NGF or another supported controller.	Customer provides controller, external address and DNS.

NGINX Gateway Fabric, F5 NGINX Ingress Controller, the retired community ingress-nginx controller, and the CNC TLS sidecar are different components. Do not use their names, configuration keys, or support status interchangeably.

12.2 NGF routing example

Install a pinned NGF version and compatible Gateway API and NGF-specific CRDs using that release's installation documentation. Gateway API's experimental bundle does not install NGF's proprietary policy resources. Do not apply several different CRD versions from unrelated examples.

YAML: `values/ngf-routing.yaml` — candidate parent-chart routing configuration

```
cim:
  gateway:
    enabled: true
    create: true
    name: "coverity-gateway"
    gatewayClassName: "nginx"
    hostnames:
      - "REPLACE_CONNECT_HOST"
    listeners:
      http:
        enabled: true
        port: 80
        redirect: true
      https:
        enabled: true
        port: 443
        tlsSecretName: "cnc-public-tls"
```

Enable snippet support only if the approved design uses NGF snippet-based allowlists. Snippets increase configuration authority; restrict who can create or modify those resources. Body limits and timeouts must match the selected controller version and the actual request path.

12.3 TLS boundaries

- The supplied Gateway documentation describes HTTP forwarding to Connect port 8080, including when Scan Services is enabled.
- Internal Scan Services-to-Connect mTLS through port 8443 is a separate path.
- NetworkPolicy restricts access but does not encrypt traffic. CNI encryption and service-mesh coverage must be evaluated separately and tested for compatibility.
- For a shared Gateway, the platform owner configures listeners, certificates, allowed namespaces, and hostname isolation. Prefer namespace selectors over unrestricted cross-namespace attachment.
- Place TLS Secrets with the Gateway for the baseline design. Cross-namespace certificate references require appropriate grants and controller support.

12.4 Traffic matrix

Source → destination	Typical port	Purpose
Browser / CI → edge	443	Connect HTTPS; optional 80 redirect.
Edge → Connect	8080 or validated TLS backend	Application traffic.
Scan Services / Jobs → Connect TLS sidecar	8443	Internal application communication where configured.
Connect / services / Jobs → Scan and Storage Services	9999; 9998 where used	gRPC and configured HTTP APIs. Do not expose publicly by default.
Analysis Jobs → Cache Service	8443	Analysis cache access.
Database clients → PostgreSQL	5432 or configured port	Application and migration connections.
Cache Service → Redis	Service-specific TLS port	Cache index.
Clients / Jobs / services → object storage	443 or configured endpoint	Artifact and toolkit transfers.
Triage API / workers → RabbitMQ	5671 TLS or configured port	Queue traffic; TLS requires matching application configuration.
Triage workers → LLM	443	Approved source-context transmission.
Workloads → DNS / Kubernetes API	53 UDP/TCP / API endpoint	Name resolution and authorized Job/Secret operations.
Nodes → image registry	443	Image pulls; not governed by ordinary pod NetworkPolicy.
Monitoring → workloads	8089 Connect; 9090 scan/storage/triage; 8081 cache	Private metrics endpoints.

12.5 Routing acceptance

Bash

```
kubectl get gateway,httproute -n "$NS"
kubectl describe gateway coverity-gateway -n "$NS"
kubectl get httproute -n "$NS" -o yaml

curl --fail --silent --show-error \
  --output /dev/null --write-out 'HTTPS: %{http_code}\n' \
  "https://$CONNECT_HOST/login/login.htm"

curl --silent --show-error --output /dev/null --dump-header - \
  "http://$CONNECT_HOST/login/login.htm"
```

Use `--cacert` with an approved private CA if needed. Do not use `-k` as an acceptance test. Verify Gateway Programmed, Route Accepted and ResolvedRefs conditions for the intended parent and current generation.

Before a DNS cutover, test using a hostname-preserving connection override such as `curl --resolve` for a known IP, rather than disabling hostname verification.

13. Analysis workers, caching, and toolkits

13.1 Dedicated worker pools

For the default small-pool profile, provision eligible nodes labeled `pool-type=small` and tainted `NodeType=ScannerNode:NoSchedule`. Configure labels and taints in the cloud node-pool template so replacement and autoscaled nodes retain them.

Application pods should have an application-node selector or affinity. A toleration permits placement; it does not require placement on those nodes. Verify the generated Job's selector, affinity, tolerations, architecture, and resources during the first test scan.

Bash: inspect placement

```
kubectl get nodes -L pool-type,kubernetes.io/arch
kubectl get pods -n "$NS" -o wide
kubectl get configmap "$RELEASE-analysis-job-config" -n "$NS" -o yaml
kubectl get jobs -n "$NS"
kubectl describe job "REPLACE_ANALYSIS_JOB" -n "$NS"
```

13.2 Custom and shared-capacity profiles

Custom CPU is in millicores and custom memory in MB in the supplied reference. For example, 4000 and 16000 request approximately 4 cores and 16 GB. Confirm actual generated quantities before using the profile.

YAML: custom-pool example

```
scan-services:
  scan-service:
    environment:
      COVANALYSIS_DEFAULTPOOLTYPE: "custom"
      CUSTOMNODEPOOL_LABEL: "custom"
      CUSTOMNODEPOOL_CPU: 4000
      CUSTOMNODEPOOL_MEM: 16000
      SMALLNODEPOOL_ENABLE: false
      MEDIUMNODEPOOL_ENABLE: false
      LARGENODEPOOL_ENABLE: false
      EXTRALARGENODEPOOL_ENABLE: false
      MULTIPLEJOBSPERNODE_ENABLE: false
```

For multiple Jobs per node, use `MULTIPLEJOBSPERNODE_ENABLE: true` and the documented shared-pool labeling for the selected profile, commonly `common-pool=scanfarm`. Validate custom-label behavior; do not assume the single-Job selector still applies.

Set cloud autoscaler minimums, maximums, quotas, node architecture, and budget limits independently. `MAXNODEPOOLSIZE` is not a replacement for provider autoscaler limits. Test scale-from-zero and interruption behavior.

13.3 Toolkit provisioning

For connected environments, optional synchronization downloads toolkits from Black Duck. It does not use an arbitrary private mirror.

Bash

```
kubectl create secret generic cnc-tool-sync -n "$NS" \
  --from-file=username=secrets/registry-username \
  --from-file=password=secrets/registry-password
```

YAML: optional toolkit synchronization overlay

```
scan-services:
  scan-service:
    tools:
      sync:
        enabled: true
        existingSecret: "cnc-tool-sync"
```

Alternatively, obtain approved `coverity-all-platforms-2026.6.0.tar.gz` artifacts and upload them through **Configuration** → **Configure Coverity Tools** → **Add Coverity Tools**. Wait for COMPLETED status before testing scans. Do not rename the archive.

- Maintain all toolkit versions used by active clients.
- Changing the default toolkit affects pipelines that use the default download URL.
- Do not archive/delete a toolkit without checking dependent pipelines and clients.
- Full Analysis client downloads require a separate licensed distribution arrangement. Do not confuse downloadable installer files with analysis Job container images.

Bash: download a pinned Linux Thin Client after it is available

```
mkdir -p client
curl --fail --location --silent --show-error \
  --output client/cov-thin-client-linux64-2026.6.0.tar.gz \
  "https://$CONNECT_HOST/api/v2/scans/downloads/cov-thin-client-linux64-2026.6.0.tar.gz"

tar -tzf client/cov-thin-client-linux64-2026.6.0.tar.gz
tar -xzf client/cov-thin-client-linux64-2026.6.0.tar.gz -C client
export PATH="$PWD/client/bin:$PATH"
coverity --help
```

Supply approved authentication if required. Inspect and verify downloaded artifacts before extraction. Configure the project using the Analysis guide; build/capture commands are language- and build-system-specific.

13.4 Caching and artifact retention

Run an initial full baseline analysis, then an eligible repeat analysis to verify cache reuse. Merely observing a healthy Cache Service pod is insufficient.

Upload mode	Retained output
All	Analyzed IDIR, analysis output, execution logs for success and failure.
OnFailure	Analyzed IDIR and execution logs on failure.
LogsOnly	Execution logs and analysis output.
None	No uploaded diagnostic artifacts; reduced troubleshooting capability.

Start acceptance testing with All, then select an approved operational policy. Keep scan retention, cache expiry, and triage retention independent. Review multipart limits, sidecar body limits, Gateway limits, and presigned URL expiration for large uploads.

14. High availability and database read replicas

HA is an end-to-end property. Multiple Connect pods do not remove database, routing, storage, or single commit-server failure points.

YAML: documented HA topology switches — requires site resource and routing review

```
cim:
  cimweb:
    replicas: 2
  commit-server:
    replicas: 2

  triage-suggestion-service:
    enabled: false
```

- Allocate the full chosen resources to every Connect pod and size commit servers separately.
- Spread replicas across failure domains using selectors that match actual pod labels, scoped to the release.
- Validate /ccd routing and every write path before production.
- Review generated Ingress resources even in Route/Gateway deployments. The parent commit-server template may create exposure independently of normal flags.
- The documented Connect strategy is Recreate. HA does not imply zero-downtime chart upgrades.
- A failed in-progress commit may need client resubmission. HA does not speed up a single commit or concurrent commits locked on the same stream.

14.1 Pgpool and read replicas

Use read replicas for measured read pressure, not simply because database CPU is high. They do not resolve a write-only bottleneck. Confirm the provider-supported replication topology and expected lag.

YAML: documented Pgpool configuration example

```
cim:
  pgpool:
    enabled: true
    maxConnections: 200
    replicadb:
      - name: "REPLACE_READ_REPLICA_1"
        port: "5432"
      - name: "REPLACE_READ_REPLICA_2"
        port: "5432"
```

The connection value is illustrative, not a sizing recommendation. Include all application, migration, monitoring, and administrative connections in the database connection budget.

The documented Connect-to-Pgpool connection is not TLS-encrypted by the chart helper. Pgpool-to-PostgreSQL TLS is separate. Review this design against the site's encryption requirements before enabling Pgpool.

Resolve the supplied AWS RDS/Aurora replication inconsistency with the vendor. Do not treat Aurora read replicas as interchangeable with RDS PostgreSQL read replicas.

15. AI-Assisted Triage

AI-Assisted Triage is optional and marked beta in the supplied 2026.6 documentation. It transmits source context and finding data to an approved LLM endpoint. Approval must cover data location, retention, logging, access, provider terms, and permitted projects.

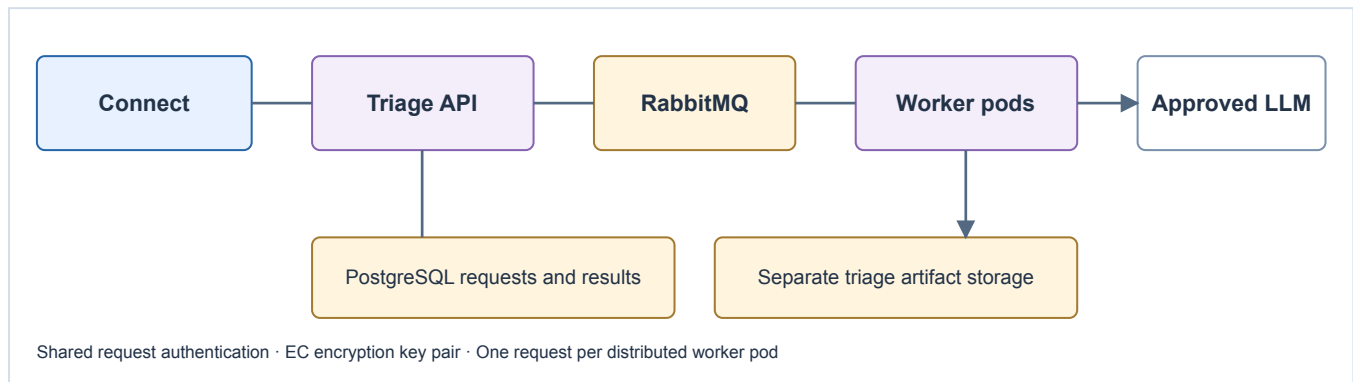


Figure 7. Triage is separate from Scan Service scheduling and analysis caching.

Activation gate: keep one Connect replica for the documented 2026.6 triage combination. Verify the actual parent and triage templates before enabling this feature. The documentation contains inconsistent Secret names, property paths, defaults, and image tags; the configuration below is a candidate, not an independently validated turnkey overlay.

15.1 Prerequisites and secrets

1. Provision the triage PostgreSQL database and migration permissions.
2. Provision RabbitMQ, credentials, vhost, queue, durable storage and TLS configuration.
3. Provide separate triage artifact storage with tested upload/download access.
4. Generate four authentication values: JWT signing secret, JWT encryption key, shared auth key, and HMAC secret. Only the shared auth key belongs on the Connect side.
5. Generate an EC P-256 encryption key pair. Mount the private key only in triage workloads and the public key in Connect.
6. Pin API and worker images to verified release tags. Verify OpenShift security contexts and supported image variants separately.

Bash: key generation in a new protected directory

```
(
  umask 077
  mkdir secrets/triage
  openssl ecparam -name prime256v1 -genkey -noout \
    -out secrets/triage/private.pem
  openssl ec -in secrets/triage/private.pem -pubout \
    -out secrets/triage/public.pem
  for name in jwt-secret jwt-encryption-key auth-key auth-hmac-secret; do
    openssl rand -base64 32 | tr -d '\n' > "secrets/triage/$name"
  done
)

kubectl create secret generic triage-suggestion-service-secrets -n "$NS" \
  --from-file=jwt-secret=secrets/triage/jwt-secret \
  --from-file=jwt-encryption-key=secrets/triage/jwt-encryption-key \
  --from-file=auth-key=secrets/triage/auth-key \
  --from-file=auth-hmac-secret=secrets/triage/auth-hmac-secret

kubectl create secret generic triage-suggestion-service-llm-key-encryption \
  -n "$NS" \
  --from-file=llm-key-private.pem=secrets/triage/private.pem \
  --from-file=llm-key-public.pem=secrets/triage/public.pem
```

The shared-key-pair Secret matches the documented wiring pattern. Verify item-level mounts so Connect receives only the public file. Where supported, separate Secrets provide stronger read-access separation.

15.2 Candidate configuration contract

Complete database credentials, RabbitMQ integration, auth-file mounts on both API and worker pods, artifact storage, and Connect authentication wiring before selecting this overlay for an install.

YAML: documented triage settings to verify against the packaged templates

```
cim:
  cimweb:
    replicas: 1
    triageSuggestionService:
      llmPublicKeySecret:
        name: "triage-suggestion-service-llm-key-encryption"
        key: "llm-key-public.pem"
        mountPath: "/secrets/triage-llm-key-encryption"

  triage-suggestion-service:
    enabled: true
    llm:
      url: "REPLACE_APPROVED_LLM_ENDPOINT"
      name: "REPLACE_SUPPORTED_MODEL"
    llmKeyEncryption:
      existingSecret: "triage-suggestion-service-llm-key-encryption"
      secretKey: "llm-key-private.pem"

  postgres:
    enabled: true
    database: "triage-suggestion-service"
    existingSecret: "cnc-triage-db"
    sslmode: "verify-ca"

  vault:
    enabled: false

  rabbitmq:
    enabled: false
```

```

externalRabbitmq:
  host: "REPLACE_RABBITMQ_HOST"
  port: 5672
  existingSecret: "cnc-triage-rabbitmq"
  existingSecretKey: "rabbitmq-password"

deployment:
  version: "2026.6.0"
  environment:
    SERVICE_MODE: "api"
    STORAGE_TYPE: "postgres"
    QUEUE_TYPE: "rabbitmq"
    MQ_USER: "REPLACE_RABBITMQ_USER"
    MQ_VHOST: "/"
    MQ_QUEUE_NAME: "triage-requests"

workerDeployment:
  version: "2026.6.0"
  environment:
    SERVICE_MODE: "worker"
    STORAGE_TYPE: "postgres"
    QUEUE_TYPE: "rabbitmq"
    MQ_USER: "REPLACE_RABBITMQ_USER"
    MQ_VHOST: "/"
    MQ_QUEUE_NAME: "triage-requests"
    WORKER_COUNT: "1"
    MQ_WORKER_CONCURRENCY: "1"

ingress:
  triage-suggestion-service-ingress:
    enabled: false

```

postgres.enabled is described here as configuration injection, not proof that a database server is deployed. The RabbitMQ example uses plain AMQP port 5672; a production TLS requirement needs both the TLS port and verified application TLS settings. Changing the port alone does not enable TLS.

For an in-cluster broker managed by a separate release, keep the bundled broker disabled and use its Service DNS as externalRabbitmq.host. “External” means external to this subchart, not necessarily external to the cluster.

15.3 Artifact storage and credentials

Back end	Documented triage settings	Credential distinction
S3	artifactStorage.storageType: s3; s3.bucket, s3.region, Secret or service account.	Verify IRSA selection in both API and worker pods.
GCS	storageType: gcs; gcs.bucket, gcs.secret.name, gcs.secret.key.	Do not assume Scan Service credentials are automatically reused.
Azure	storageType: azure; container, storage account and Secret.	Documented key is azure_storage_key, different from Scan Storage Service's azure_account_key.
MinIO	storageType: minio; bucket, endpoint, secure, pathStyle, Secret.	root-user and root-password; validate TLS trust.

Review effective retention. The runtime reference and Helm examples show different request/result/artifact durations. Select explicit approved values and verify the rendered environment. Do not rely on a single “default retention” claim.

15.4 LLM keys, scaling, and acceptance

- Use the documented authenticated Connect APIs or UI workflow to register project/global LLM credentials. Avoid direct database edits.
- Verify the selected `useGlobalKey` behavior. Project-specific mode requires the appropriate project key.
- Keep distributed `WORKER_COUNT=1` and `MQ_WORKER_CONCURRENCY=1`. Scale worker pods rather than increasing per-pod request concurrency.
- Review API/worker HPA minimums and maximums, metrics availability, database connection totals and LLM rate limits.
- Test liveness, readiness, shared authentication, encryption/decryption, queue processing, result retrieval, and cleanup using an approved supported finding.
- Do not use development Vault or fixed development root tokens. Disable unused Vault integration explicitly and inspect all inherited environment settings.
- Rotate the shared auth key on both Connect and triage in a coordinated window.

Sources: PDF pp.4146–4168, 4470–4504.

16. Validation and installation

16.1 Select exactly one path

Path	Values file sequence
Connect-only with NGF	base → ngf-routing → site
AWS cached analysis with NGF	base → scan → aws-storage → ngf-routing → site
Azure cached analysis with NGF	base → scan → azure-storage → ngf-routing → site
GCP cached analysis with native Gateway	base → scan → gcp-storage → gcp-routing → site
OpenShift cached analysis	base → scan → openshift-platform → openshift-storage → site

Other routing implementations require their reviewed controller-specific configuration. Add HA or triage only after completing those sections and their compatibility gates.

Bash: example AWS selection; change the list for the chosen path

```
FILES=(
  values/base.yaml
  values/scan.yaml
  values/aws-storage.yaml
  values/ngf-routing.yaml
  values/site.yaml
)

HELM_VALUES=()
for file in "${FILES[@]"; do
  test -f "$file" || { echo "Missing $file"; exit 1; }
  HELM_VALUES+=(-f "$file")
done

if grep -n 'REPLACE_' "${FILES[@]"; then
  echo "Replace all selected-file placeholders before proceeding."
  exit 1
fi

export KUBE_VERSION="REPLACE_ACTUAL_SERVER_VERSION"

API_FLAGS=()
# OpenShift only:
# API_FLAGS+=(--api-versions security.openshift.io/v1)
# API_FLAGS+=(--api-versions route.openshift.io/v1)
# Add the installed Gateway/provider API versions if chart logic requires them.
```

16.2 Render and inspect

Bash

```
helm lint "$CHART" "${HELM_VALUES[@]}" \
  --set-file cim.ldap.cimKey.value=secrets/ldap-key

helm template "$RELEASE" "$CHART" \
  --namespace "$NS" --kube-version "$KUBE_VERSION" \
  "${API_FLAGS[@]}" "${HELM_VALUES[@]}" \
  --set-file cim.ldap.cimKey.value=secrets/ldap-key \
  > review/rendered.yaml

chmod 600 review/rendered.yaml

kubectl apply --dry-run=server -n "$NS" \
  -f review/rendered.yaml
```

Server dry-run validates objects but does not execute hooks or validate runtime connectivity. Offline rendering cannot fully model lookup behavior. Use Helm for installation; applying rendered hooks directly bypasses their lifecycle.

Required review checklist

- No placeholders, unintended services, development images, or unexpected public endpoints.
- Correct Secret keys, ConfigMap names, database names, storage adapters and identity bindings.
- Database creation flags off for precreated databases; required migration hooks on.
- Migration-log credential exposure corrected or covered by an approved mitigation.
- Correct image architecture and tags for every container and dynamic Job.
- Correct public URL, HTTPS storage origins, commit URL and sidecar/backend ports.
- Resources fit quotas and individual nodes, including hook and rollout peaks.
- All referenced CRDs/controllers installed at compatible pinned versions.
- Security contexts, projected tokens and RBAC meet the chosen platform's policy.

16.3 Install a new release

Complete all gates and prerequisites first. A restored database requires the migration preparation in section 18 before this command.

Bash

```
helm install "$RELEASE" "$CHART" \
  --namespace "$NS" \
  "${HELM_VALUES[@]}" \
  --set-file cim.ldap.cimKey.value=secrets/ldap-key \
  --wait --timeout 60m

helm status "$RELEASE" -n "$NS"
kubectl get deployments,statefulsets,pods,jobs,services -n "$NS"
```

Do not use automatic rollback as a substitute for database recovery planning. Hooks may have changed database schemas even if installation fails.

16.4 Observe readiness

Bash

```
kubectl rollout status "deployment/$RELEASE-cim" \  
-n "$NS" --timeout=15m  
  
# Scan-enabled profiles:  
kubectl rollout status "deployment/$RELEASE-scan-service" \  
-n "$NS" --timeout=15m  
kubectl rollout status "deployment/$RELEASE-storage-service" \  
-n "$NS" --timeout=15m  
  
# Caching profile:  
kubectl rollout status "deployment/$RELEASE-cache-service" \  
-n "$NS" --timeout=15m  
  
kubectl get jobs -n "$NS"  
kubectl get events -n "$NS" --sort-by=.metadata.creationTimestamp
```

Use actual rendered names if the package differs. Check setup, license, certificate, password and database migration Jobs individually. Do not distribute raw migration logs until they have been reviewed for credentials.

17. Acceptance testing and integrations

Test	Pass condition
Release identity	Expected chart and images; no moving tags or unexpected components.
Infrastructure	Pods ready, hooks complete, PVCs bound, service endpoints populated.
HTTPS	Trusted certificate, correct hostname, expected redirect, approved backend protocol.
Authentication	Administrator and ordinary-user login; correct role/project access.
SSO	SAML/LDAP works with canonical HTTPS URL; break-glass access retained.
Connect-only commit	Full client commits results; snapshot, source and cross-references visible.
Cloud analysis	Capture upload → Job scheduling → analysis → commit → artifact download succeeds.
Large transfer	Representative large IDIR/toolkit upload completes without timeout, expiry or body-limit errors.
Cache	Baseline and eligible repeat scan demonstrate expected cache use.
Scaling	Jobs land on intended workers; autoscaling creates usable capacity within quota.
AI triage	Approved finding completes authentication, queueing, inference and result retrieval.
Monitoring	All required targets scrape successfully and alerts reach the owning team.
Recovery	Backup restoration tested; configuration and keys recoverable.

Configure SAML and first-time LDAP through Connect's supported UI procedures. For TLS integrations, add approved CA trust. Test SMTP, Jira, Bugzilla or other configured integrations independently.

Do not alter the default toolkit for all pipelines until representative users have passed acceptance. Record the scan IDs, versions, duration, workload, resource observations, and approver.

Bash: workload endpoint inspection

```
kubectl get endpointslices.discovery.k8s.io -n "$NS" \
  -l "kubernetes.io/service-name=$RELEASE-cim"

kubectl logs "deployment/$RELEASE-cim" -n "$NS" \
  -c cim-webapp --tail=200

kubectl get pods -n "$NS" -o wide
kubectl get pvc -n "$NS"
```

18. Migration, upgrades, and recovery

18.1 Migration rehearsal

1. Inventory source versions, database engine, size, integrations, client versions, custom configuration, certificates, licenses, and encryption keys.
2. Confirm the supported application upgrade path and any PostgreSQL major-version transition.
3. Back up the source and restore into an isolated target using the DBA-approved process.
4. Preserve `cim.ldap.key`. Preserve triage keys and credentials if that feature is already in use.
5. Disable production notifications and external writes in the rehearsal environment.
6. Run the chart setup/migration process against the restored target and measure duration.
7. Verify representative historical data, permissions, integrations, scans and performance.
8. Rehearse recovery to the original application/database combination.

When migrating an existing full CNC deployment, include scan/storage metadata and corresponding object storage. Restoring only PostgreSQL while losing referenced objects does not restore a complete platform.

18.2 Production cutover

1. Pause CI submissions, commits, API writers, integrations, and new triage requests.
2. Drain or explicitly cancel analysis/triage work under the approved procedure.
3. Stop every writer to the affected databases; verify that autoscalers and GitOps controllers cannot restart them unexpectedly.
4. Take final consistent backups and transfer any required object data.
5. Restore and run the rehearsed migration/upgrade sequence.
6. Complete acceptance before changing DNS or resuming production traffic.
7. Resume a limited workload, observe it, then resume normal operation.

18.3 Helm upgrade

Build a complete reviewed values set for the target release. Do not blindly reuse old values, stale keys, or archived dependency overrides. Keep the original release package for recovery.

Bash: protected configuration capture

```
helm get values "$RELEASE" -n "$NS" \
  > review/before-values.yaml
helm get manifest "$RELEASE" -n "$NS" \
  > review/before-manifest.yaml
helm get hooks "$RELEASE" -n "$NS" \
  > review/before-hooks.yaml
chmod 600 review/before-*.yaml
```

For Connect-only, scale the application down and confirm termination before the upgrade. For scan-enabled or HA deployments, follow the full writer-shutdown sequence instead of scaling only Connect.

Bash: after backups and writer shutdown

```
helm upgrade "$RELEASE" "$CHART" \
  --namespace "$NS" \
  --reset-values \
  "${HELM_VALUES[@]}" \
  --set-file cim.ldap.cimKey.value=secrets/ldap-key \
  --wait --timeout 60m
```

- A nonempty administrator password Secret reference may reset the password through the upgrade hook. Remove it unless a reset is intended.
- Preserve RabbitMQ passwords and Erlang cookies with persistent broker data.
- Do not toggle a database or broker dependency off and on as a blind upgrade repair.
- Inspect deprecated/renamed values across every intervening release.
- Restore normal replicas, autoscaling and GitOps operation only after schema and application checks pass.

Rollback boundary: Helm rollback does not undo database schema changes. Recover with a compatible database backup and application version. After the target accepts new writes, switching back requires an explicit data-loss/reconciliation decision. Never reconnect old binaries to an upgraded database without confirmed compatibility.

18.4 Recovery record

Record backup timestamps, restore commands/process, database versions, object-store recovery point, chart digest, values, keys, DNS reversal, acceptance criteria, and the named recovery decision owner. Test restoration regularly, not only immediately before an upgrade.

19. Operations and troubleshooting

19.1 Monitoring

Enabling metrics does not install Prometheus or Grafana. Configure discovery, network access, retention, dashboards and alert delivery. Verify actual labels before copying PromQL selectors.

Signal	Action
Growing commit queue	Compare active commits, executor size, stream locks, database CPU and I/O. Increasing queue size alone is not a fix.
Long scan dispatch time	Check pending Jobs, selectors, taints, node allocatable resources, autoscaler limits and cloud quotas.
Sustained high memory / OOM	Review pod limits, JVM/native usage, IDIR sizes and concurrency. Preserve diagnostics.
Redis noeviction errors	Check memory and cache growth; increase capacity or correct lifecycle behavior under the approved design.
Database/storage growth	Review retention, WAL, backups, toolkit versions, object data and maintenance schedules.
Triage backlog	Check worker health, RabbitMQ, LLM rate limits, credential expiry and worker HPA.
Certificate expiry	Track public certificates, internal CA/leaf certificates, database and Redis CA changes separately.

19.2 Common diagnostics

Bash

```
kubectl get events -n "$NS" --sort-by=.metadata.creationTimestamp
kubectl get deployments,statefulsets,pods,jobs,services,pvc -n "$NS"
kubectl describe deployment "$RELEASE-cim" -n "$NS"
kubectl describe pod "REPLACE_POD" -n "$NS"

kubectl logs "deployment/$RELEASE-cim" -n "$NS" \
  -c cim-webapp --tail=200

kubectl logs "deployment/$RELEASE-scan-service" -n "$NS" --tail=200
kubectl logs "deployment/$RELEASE-storage-service" -n "$NS" --tail=200
```

Symptom	Check first
ImagePullBackOff	Full path/tag, architecture, node registry access, credential expiry, pull Secret and registry CA.
Migration fails	Database existence, owner/extension permissions, TLS trust, resource limits and source schema. Treat logs as sensitive.
Pending pod	Individual node capacity, quotas, selectors, affinity, taints, PVC topology and admission policy.
503 / unhealthy backend	EndpointSlices, readiness, actual service port, health-check path and controller status.
413 / upload timeout	Locate the rejecting hop: edge, sidecar, application, object-store proxy or signed URL expiry.

Symptom	Check first
Browser CORS/CSP failure	Canonical HTTPS origins, storage CORS, custom storage domain policy and signed endpoint reachability.
Scan queued indefinitely	Toolkit availability, Job creation RBAC, worker labels/resources, registry access and scheduler events.
Internal certificate error	Certificate dates, SANs, Secret names, CA consistency and workload reload behavior.
AI triage 401/403	Shared auth key, file mounts, public/private key pairing and selected LLM credential scope.

pg_isready checks server connection status; it does not prove correct password authentication, required permissions, or application TLS behavior.

19.3 Administrative tools

Bash

```
kubectl scale "statefulset/$RELEASE-cim-tools" \
  -n "$NS" --replicas=1

kubectl rollout status "statefulset/$RELEASE-cim-tools" \
  -n "$NS" --timeout=5m

kubectl exec -it -n "$NS" "statefulset/$RELEASE-cim-tools" \
  -- /coverity/shell-entrypoint.sh
```

Inside the tools shell, use cloud-specific scripts:

Bash

```
check-integrity.sh
# For an approved administrator password reset:
# reset-admin-password.sh

# Example approved stream export:
# ./cov-archive.sh export-streams \
#   --archive /workdir/project.archive \
#   --project "REPLACE_PROJECT" --stream "REPLACE_STREAM"

exit
```

Copy required files before scaling down:

Bash

```
# After an export exists:
# kubectl cp -n "$NS" \
#   "$RELEASE-cim-tools-0:/workdir/project.archive" \
#   review/project.archive

kubectl scale "statefulset/$RELEASE-cim-tools" \
  -n "$NS" --replicas=0
```

/workdir is temporary in the reviewed runbook design. Use an approved tools PVC for persistence. Confirm volume names and mount paths in the actual chart. Archive import requires the documented application shutdown and locking procedure; copying an archive into a pod does not import it.

Do not use classic `cov-admin-db` backup, restore, optimize, `psql`, or scramble as CNC database administration procedures. Use the supported cloud tools and DBA backup process.

19.4 Database tuning

Start with the release's tuning-suggest workflow. Review recommendations with the DBA, compare them with provider constraints and total server workload, and schedule any required restart. Tuning-write requires additional provider privileges; do not give them permanently to application runtime identities.

Table rewrites, full vacuum, ETL truncation, and cross-reference deduplication are separate maintenance activities with locking, data-retention and recovery implications. Do not run them as generic troubleshooting commands.

19.5 Certificate and credential rotation

1. Inventory affected consumers and establish a maintenance/recovery plan.
2. Back up necessary configuration and protected key material.
3. For internal certificate renewal, identify the exact cert-generation hook by resource name and kind. Do not select the first Job in an arbitrary hooks file.
4. Verify the certificate manifest, hook RBAC and image before running regeneration.
5. Replace only the intended certificate Secrets under the approved procedure; restart/reload all affected consumers and verify mTLS.
6. Rotate public Route certificate material separately from internal service PKI.
7. After database or broker password rotation, verify both running consumers and future hook/Job credentials.

19.6 Decommissioning

Stop writers, retain required backups and exports, obtain deletion approval, then uninstall the specific release. Inventory remaining hooks, Secrets, PVCs, buckets, DNS and cloud resources. Delete persistent data only through a separate approved retention/decommission process. Never use namespace-wide deletion as a routine cleanup shortcut.

20. Evaluation and disconnected environments

20.1 Evaluation

Use the release-matched CNC quick-start material for documented lab targets such as Kind. Local clusters are for learning and functional checks, not proof of production scalability or availability.

- Provide enough host/container-runtime resources for the selected profile.
- Local storage and a single node are failure points.
- A local LoadBalancer Service may require a supported local implementation or explicit port mapping.
- A reduced-resource demo is not the published production minimum.
- Do not carry example passwords, HTTP-only exposure, development Vault, or floating tags into production.

20.2 Disconnected operation

1. Inventory and mirror the CNC package, all dependencies, controllers, CRDs, application images, hook images, Job images, toolkits and required client packages.
2. Verify every architecture and tag; capture digests and checksums.
3. Disable toolkit synchronization and provision toolkits through the approved offline transfer and Connect upload process.
4. Configure internal DNS, PKI, time synchronization and trust distribution.
5. Test installation and an end-to-end scan without external egress.
6. For AI triage, provide an approved reachable LLM endpoint or keep the feature disabled. An air gap does not eliminate the LLM dependency.

A proxy is not a replacement for an offline artifact inventory. If an outbound proxy is used, distinguish node image-pull proxy configuration from application proxy settings and protect proxy credentials/certificates.

21. References and validation record

21.1 Source map

Primary source: **Coverity Documentation — New Fluid Topics Tenant.pdf**, Coverity 2026.6 Cloud Deployment Administrator and User Guide. Page numbers below are 1-based PDF positions, not HTML chapter numbers.

Subject	PDF pages
Architecture and deployment scenarios	3751–3800
AWS	3853–3875
GCP	3875–3887
Azure	3888–3903
OpenShift	3903–3914
PostgreSQL and tuning	3915–3950
Gateway API and provider routing	3950–4004
Connect sizing	4004–4015
Certificates, Secrets and configuration	4015–4096
Security and scheduling	4096–4119
Bundled dependencies and downloads	4119–4138
HA	4138–4146
AI triage	4146–4168; 4470–4504
Installation, clients and metrics	4168–4210
Upgrades, troubleshooting and support	4239–4295
Scan Services values	4401–4469
Cloud tools	4504–4512

21.2 Authoritative implementation references

- Black Duck Community: <https://community.blackduck.com/>
- CNC chart: `oci://repo.blackduck.com/cloudnative/cnc`
- Application registry: `repo.blackduck.com/containers`
- Helm: <https://helm.sh/docs/>
- Gateway API: <https://gateway-api.sigs.k8s.io/>
- NGINX Gateway Fabric: <https://docs.nginx.com/nginx-gateway-fabric/>
- EKS: <https://docs.aws.amazon.com/eks/latest/userguide/>
- RDS PostgreSQL: [AWS RDS PostgreSQL documentation](#)
- AKS: <https://learn.microsoft.com/en-us/azure/aks/>

- Application Gateway for Containers: [Microsoft AGfC documentation](#)
- GKE Gateway: [Google GKE Gateway documentation](#)
- Cloud SQL PostgreSQL: <https://cloud.google.com/sql/docs/postgres>
- OpenShift: [Red Hat OpenShift documentation](#)

21.3 Site validation record

Evidence	Required record
Supported combination	Product/platform versions and support confirmation reference.
Release package	Chart archive checksum, subchart versions, image digests.
Configuration	Selected file order, reviewed values revision and rendered-resource review.
Security approval	TLS boundaries, RBAC, network policies, secrets handling, migration-log mitigation and optional AI data approval.
Functional acceptance	Scan/commit IDs, toolkit version, integrations, signed transfers and cache test.
Capacity	Representative load, resources, latency, throughput, scaling and headroom.
Recovery	Backup and restoration evidence, RPO/RTO, preserved keys and rollback decision.
Operations	Monitoring targets, alert recipients, certificate and credential rotation owners.
Document quality	Exported PDF inspected for diagram labels, table headers, page breaks, clipped commands and working internal links.

To export: browser Print → Save as PDF, portrait, A4, 100% scale. Disable browser-generated headers/footers. Inspect the output before distribution. For Letter paper, adjust the page size and recheck pagination.

Coverity 2026.6 — Multiplatform Deployment Engineering Guide

Release-pinned reference. Preserve approved configuration, release artifacts, support decisions, verification evidence and recovery procedures with the deployment change record.